

HW2

Pyro를 활용하여, immigrants 자료(링크)에 대해 베이지안 회귀모형을 적용하는 코드와 분석결과를 작성하세요.

1. 변분추론을 이용해야 합니다.
2. immigrants 자료에서 wage를 Y(종속변수)로, 나머지 세 개의 변수 sp, lit, n_live를 X(독립변수)로 두고 회귀모형을 적용하세요.

베이지안 회귀모형을 적용한다.

Model 은 다음과 같다.

$$y \sim N(X\beta, \frac{1}{\tau} I_n)$$

where $y = (y_1, \dots, y_n)^T$, $X \in \mathbb{R}^{N \times p}$ is the design matrix, $\beta = (a, b_{sp}, b_{lit}, b_{nlive})^T$

사전분포 정의

절편과 각 회귀계수 $a, b_{sp}, b_{lit}, b_{nlive}$ 가 모두 $N(0, 10^2)$ 을 따른다고 가정한다.
이는 계수의 부호와 크기에 대해 강한 사전 정보를 두지 않는 약한 사전분포이다.

$$\beta \sim N(0, 10^2 I_p)$$

분산의 역수인 정밀도 τ 는 shape와 rate가 각각 0.01인 감마분포를 따른다고 가정한다.
이는 τ 가 양수라는 조건만 반영하면서, 오차분산의 크기에 대해 강한 사전정보를 두지 않는 약한 사전분포이다.

$$\tau \sim Ga(0.01, 0.01)$$

변분분포

$$q_{\lambda}(\theta) = N(\beta | \mu_{N,\beta}, \sigma_{N,\beta}^2) Ga(\tau | a_{N,\tau}, b_{N,\tau}),$$

where $\theta = (\beta, \tau)^T$ and $\lambda = (\mu_{N,\beta}, \sigma_{N,\beta}^2, a_{N,\tau}, b_{N,\tau})$

```
In [12]: import jax
import pandas as pd

# 데이터
dset = pd.read_csv('immigrants.csv', index_col=0)
dset.head()
```

```
Out[12]:
```

	nationality	wage	sp	lit	n_live
1	Armenian	9.73	54.9	92.1	54.6
2	Bohemian/Moravian	13.07	66.0	96.8	71.2
3	Bulgarian	10.31	20.3	78.2	8.5
4	Canadian(French)	10.62	79.4	84.1	86.7
5	Canadian(Other)	14.15	100.0	99.0	90.8

```
In [13]: # 데이터 표준화 (평균 0, 표준편차 1)
standardize = lambda x: (x - x.mean()) / x.std()

dset['wageScaled'] = dset.wage.pipe(standardize)
dset['spScaled'] = dset.sp.pipe(standardize)
dset['litScaled'] = dset.lit.pipe(standardize)
dset['n_liveScaled'] = dset.n_live.pipe(standardize)
```

```
In [14]: # 모형정의 - likelihood, prior
def regmodel(sp=None, lit=None, n_live=None, wage=None):
    # prior
    a = numpyro.sample('a', dist.Normal(0, 10)) # 절편
    b_sp = numpyro.sample('b_sp', dist.Normal(0, 10)) # 기울기 sp
    b_lit = numpyro.sample('b_lit', dist.Normal(0, 10)) # 기울기 lit
    b_nlive = numpyro.sample('b_nlive', dist.Normal(0, 10)) # 기울기 n_live
    mu = a + b_sp * sp + b_lit * lit + b_nlive * n_live

    tausq = numpyro.sample('tausq', dist.Gamma(0.01, 0.01))
    sigma = jax.numpy.sqrt(1.0 / tausq)

    # Likelihood
    with numpyro.plate('data', len(wage)):
        numpyro.sample('obs', dist.Normal(mu, sigma), obs=wage)
```

```
In [15]: # 변분분포 정의
def guide(sp=None, lit=None, n_live=None, wage=None):
    # tau
    alpha_tau = numpyro.param('alpha_tau', 1.0, constraint=dist.constraints.positive)
    beta_tau = numpyro.param('beta_tau', 1.0, constraint=dist.constraints.positive)
    numpyro.sample('tausq', dist.Gamma(alpha_tau, beta_tau))

    # a
    loc_a = numpyro.param('loc_a', 0.0)
    scale_a = numpyro.param('scale_a', 1.0, constraint=dist.constraints.positive)
    numpyro.sample('a', dist.Normal(loc_a, scale_a))

    # b_sp
    loc_sp = numpyro.param('loc_sp', 0.0)
    scale_sp = numpyro.param('scale_sp', 1.0, constraint=dist.constraints.positive)
    numpyro.sample('b_sp', dist.Normal(loc_sp, scale_sp))

    # b_lit
    loc_lit = numpyro.param('loc_lit', 0.0)
    scale_lit = numpyro.param('scale_lit', 1.0, constraint=dist.constraints.positive)
    numpyro.sample('b_lit', dist.Normal(loc_lit, scale_lit))

    # b_nlive
    loc_nlive = numpyro.param('loc_nlive', 0.0)
    scale_nlive = numpyro.param('scale_nlive', 1.0, constraint=dist.constraints.positive)
    numpyro.sample('b_nlive', dist.Normal(loc_nlive, scale_nlive))
```

```
In [18]: # 최적화
import numpyro
# import numpyro.distributions as dist
from numpyro.infer import SVI, Trace_ELBO
from numpyro.optim import Adam

# 최적화를 위한 Adam 옵티마이저 설정
optimizer = Adam(step_size=0.001)

# SVI 객체 생성
svi = SVI(regmodel, guide, optimizer, loss=Trace_ELBO())

# SVI 실행
n_steps = 10000
```

```

svi_state = svi.init(jax.random.PRNGKey(0),
                     sp=dset.spScaled.values,
                     lit=dset.litScaled.values,
                     n_live=dset.n_liveScaled.values,
                     wage=dset.wageScaled.values)

# 최적화 루프
elbo_values = []
for step in range(n_steps):
    svi_state, elbo = svi.update(svi_state,
                                sp=dset.spScaled.values,
                                lit=dset.litScaled.values,
                                n_live=dset.n_liveScaled.values,
                                wage=dset.wageScaled.values)

    elbo_values.append(-elbo)
    if step % 1000 == 0:
        print('step {}: ELBO = {}'.format(step, elbo))

```

```

step 0: ELBO = 89.01730346679688
step 1000: ELBO = 90.34576416015625
step 2000: ELBO = 64.99374389648438
step 3000: ELBO = 74.2669448852539
step 4000: ELBO = 67.7530517578125
step 5000: ELBO = 69.41201782226562
step 6000: ELBO = 46.484920501708984
step 7000: ELBO = 47.53099822998047
step 8000: ELBO = 52.57243728637695
step 9000: ELBO = 46.74317169189453

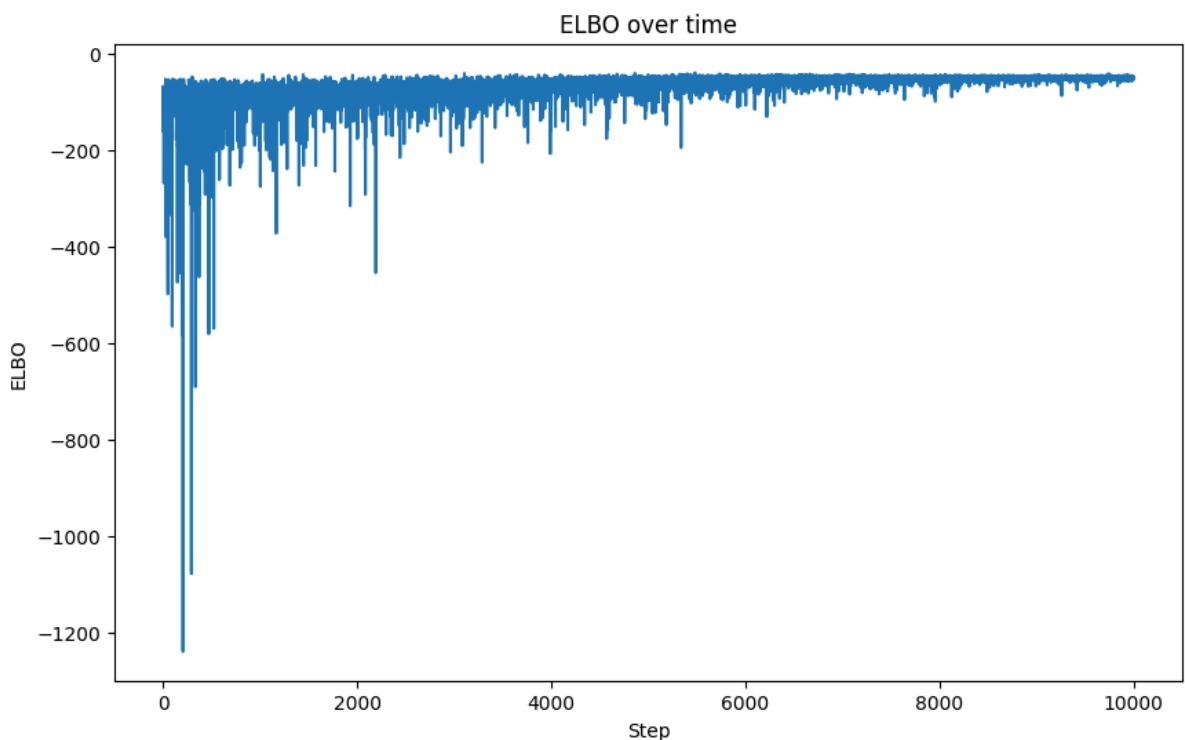
```

In [20]: `import matplotlib.pyplot as plt`

```

# 수렴성 판단
plt.figure(figsize=(10, 6))
plt.plot(elbo_values)
plt.title('ELBO over time')
plt.xlabel('Step')
plt.ylabel('ELBO')
plt.show()

```



In [26]: `# 회귀식 및 변수 유의성 요약`
`import pandas as pd`

```

params = svi.get_params(svi_state)

coef_summary = pd.DataFrame([
    ["절편", params["loc_a"], params["scale_a"]],
    ["sp", params["loc_sp"], params["scale_sp"]],
    ["lit", params["loc_lit"], params["scale_lit"]],
    ["n_live", params["loc_nlive"], params["scale_nlive"]],
], columns=["변수", "계수", "표준편차"])

coef_summary["계수"] = coef_summary["계수"].astype(float)
coef_summary["표준편차"] = coef_summary["표준편차"].astype(float)
coef_summary["95% 하한"] = coef_summary["계수"] - 1.96 * coef_summary["표준편차"]
coef_summary["95% 상한"] = coef_summary["계수"] + 1.96 * coef_summary["표준편차"]
coef_summary["유의성"] = coef_summary.apply(
    lambda x: "유의" if x["95% 하한"] > 0 or x["95% 상한"] < 0 else "비유의",
    axis=1
)

a, b_sp, b_lit, b_nlive = coef_summary["계수"]

print(
    f"회귀식: wage_scaled = {a:.3f} "
    f"+ ({b_sp:.3f})sp_scaled "
    f"+ ({b_lit:.3f})lit_scaled "
    f"+ ({b_nlive:.3f})n_live_scaled"
)

print("\n핵심 해석:")
for _, row in coef_summary.iloc[1:].iterrows():
    print(
        f"{row['변수']}: 계수 {row['계수']:.3f}, "
        f"95% 구간 [{row['95% 하한']:.3f}, {row['95% 상한']:.3f}] → {row['유의성']}"
    )

```

회귀식: $\text{wage_scaled} = -0.002 + (0.392)\text{sp_scaled} + (0.516)\text{lit_scaled} + (0.032)\text{n_live_scaled}$

핵심 해석:

sp: 계수 0.392, 95% 구간 [0.162, 0.621] → 유의

lit: 계수 0.516, 95% 구간 [0.300, 0.731] → 유의

n_live: 계수 0.032, 95% 구간 [-0.184, 0.248] → 비유의

각 회귀계수의 95% 구간을 기준으로 변수의 유의성을 판단하였다.

- sp의 계수는 0.392이고 95% 구간은 [0.162, 0.621]이다. 0을 포함하지 않으므로 wage에 유의한 양의 영향을 준다고 볼 수 있다.
- lit의 계수는 0.516이고 95% 구간은 [0.300, 0.731]이다. 0을 포함하지 않으므로 wage에 유의한 양의 영향을 준다고 볼 수 있다.
- n_live의 계수는 0.032이고 95% 구간은 [-0.184, 0.248]이다. 0을 포함하므로 유의한 영향이 있다고 보기 어렵다.

따라서 immigrants 자료에서는 sp와 lit가 wage를 설명하는 데 중요한 변수로 나타났으며, n_live는 통계적으로 뚜렷한 효과를 보이지 않았다.